DESIGN SPECIFICATION - IMPLEMENTATION GATED

CourierLedger M1
Database Design Specification

PostgreSQL database architecture for a Bangladesh-first, multi-tenant COD reconciliation service for small and medium online sellers. CSV-first for Steadfast, Pathao, Paperfly, RedX and eCourier.

Product	CourierLedger
Document status	Review draft for senior backend and database review. Not an implementation authorisation.
Financial authority	Specification V2, V2.1, V2.2 and V2.2.1 amendments, plus the P0 final logic consistency and safety patch. Where this document and V2.2.1 differ on a formula, V2.2.1 prevails.
Stack	PostgreSQL, Python + FastAPI, SQLAlchemy 2.x, Alembic, modular monolith, React + TypeScript + Vite, Docker application with managed PostgreSQL.
Tenancy model	Shared database, shared schema, PostgreSQL Row-Level Security (RLS) with FORCE, composite tenant foreign keys and a non-owner runtime role.
Gate	No production implementation, migration or deployment is authorised by this document. M1 remains gated behind market validation (Day 7 and Day 14 gates) and explicit production-readiness approval.

> **Read this first.** This specification designs the database only. It contains no production application code and no executable migration. Illustrative SQL in Section E must be adapted, reviewed and tested before any production use. No courier rate, fee base or statement format is asserted here: every courier-specific rule must come from merchant-confirmed evidence recorded in `courier_rule_evidence`.

What this document decides, and what it does not

It settles the table structure, tenant isolation model, financial storage rules, snapshot and immutability design, indexing approach, lifecycle defaults and test scenarios needed for a reviewer to approve or reject the M1 database before any code is written. It does not settle legal retention periods, authentication provider choice, file-storage provider choice, or any individual merchant's courier contract. Those are listed as open decisions in Section J, each with a recommended default and a note on whether it blocks implementation.

Contents

How to read this document - labels, conventions and notation

A. Architecture Review

B. Detailed Entity Relationship Diagram (Mermaid)

C. Table-by-Table Specification

D. Financial Data Integrity

E. PostgreSQL RLS Design

F. Indexing and Performance

G. Data Lifecycle and Retention

H. SQLAlchemy 2.x and Alembic Design

I. Database Testing Strategy

J. Open Decisions

Appendices - invariant traceability, glossary, review checklist

How to read this document

Labels used throughout

Label	Meaning
CONFIRMED	Stated in the approved baseline, V2.2.1 or the P0 patch, or in the task brief for this specification. Not open to silent change in implementation.
RECOMMENDED	A design recommendation in this document. A reviewer may reject it, but the rejection and its replacement must be recorded before implementation.
OPEN	Unresolved. Listed again in Section J with options and a recommended default.
ADAPT	Illustrative SQL or configuration that must be adapted and tested before production use. Never copy it verbatim into a live system.

Conventions

- **Money:** every amount column is `BIGINT` integer poisha (100 poisha = 1 taka). The suffix `_poisha` is mandatory in column names. No `FLOAT`, `REAL`, `DOUBLE PRECISION` or `NUMERIC` money column is permitted in M1; rates and percentages are not money and use the types stated per table.
- **Identifiers:** primary keys are `UUID` generated by the application (or `gen_random_uuid()` where stated). Natural identifiers such as tracking IDs are never primary keys.
- **Time:** all timestamps are `TIMESTAMPTZ`, stored in UTC, displayed in Asia/Dhaka by the application.
- **NULL:** NULL means unknown, missing, not applicable as defined per column, or not yet calculable. NULL never means zero. Verified zero is the integer `0` plus, where a source value is involved, a data-quality state of `explicit_zero`.
- **Tenant:** the tenant is the merchant workspace. The tenant column is `merchant_id` on every tenant-owned table, including tables reached only through a parent.
- **Immutability classes:** *Append-only* (no UPDATE, no DELETE for the runtime role), *Versioned* (changes create a new row; the old row is closed, never edited in place except for its validity end), and *Mutable* (ordinary updates, audited).

A. Architecture Review

A.1 Verdict on the draft

The proposed module list is a sound starting point. Its module boundaries match the domain - identity, courier configuration, orders, import, reconciliation and settlement are genuinely different lifecycles - and it already separates raw import data, reconciliation output and bank settlement, which is the separation the financial specification depends on. It should not be adopted unchanged. Five entities are missing, two entities need restructuring, and the draft as written does not by itself deliver tenant isolation, statement-line traceability or per-fee-line rounding evidence.

> **Summary of changes to the draft.** Add `statement_lines`, `reconciliation_fee_lines`, `shipment_match_candidates`, `file_objects` and `idempotency_keys`. Restructure `column_mappings` so a mapping belongs to a statement template and each import stores an immutable snapshot of the mapping actually used. Keep `import_errors`, but define it as a queryable projection of row validation, not a second source of truth. Keep `workspace_settings` as a strict one-to-one table rather than a JSON column on `merchants`, because tolerance is a typed financial control. No draft entity is deleted.

A.2 What the draft gets right

- Rule versions and rule evidence are separate tables. That is correct and essential: a version states what was calculated, evidence states why anyone is entitled to believe it.
- Reconciliation is modelled as runs, results and findings rather than columns on shipments. That preserves run immutability and lets findings coexist without netting.
- Settlement separates batches, receipts and allocations. That three-way split is what makes partial bank receipts representable without inventing a variance.
- Import separates batches, rows and errors. Raw rows can therefore survive a failed or disputed reconciliation.
- Audit and settings are first-class, not an afterthought bolted onto user records.

A.3 Missing entities, and why each is needed

Added entity	Why the draft cannot work without it
`statement_lines`	The draft jumps from raw `import_rows` to reconciliation results. There is nowhere to store the normalised courier-reported values for one shipment - reported collected COD, reported fees, reported payout - with their per-field data-quality state. Without this table, reported values would have to be parsed again at reconciliation time (so a run could not snapshot its inputs) or stored on `shipments` (so a second statement would overwrite the first). **RECOMMENDED, blocking.**
`reconciliation_fee_lines`	Rounding is per fee line, and the specification requires expected-basis and reported-basis fees to be shown separately. A single totals row cannot prove which line rounded, which line was NULL, or why `X_e`, `X_r` and `X_R` differ. One row per fee component per result carries that evidence.
`shipment_match_candidates`	Weak matching must be auditable and must never rely on amount alone. If only the final chosen match is stored, there is no record that a match was ambiguous, which candidates competed, or why a human confirmed one. This table stores every candidate, its match basis, its rank and its confirmation state.
`file_objects`	An import batch needs a verifiable link to the uploaded file: content hash, size, storage key and upload state. Keeping only a filename on the batch makes duplicate-file detection and later re-verification impossible.
`idempotency_keys`	CSV upload and run creation will be retried by browsers, networks and users. Without stored idempotency keys, a retry creates a second batch or a second run and the duplicate looks like new evidence.

A.4 Entities restructured, and entities questioned

Entity	Finding
`column_mappings`	**Restructure.** A mapping is a property of a statement template version, not a free-standing object, and an import must not change when a template is later edited. So `column_mappings` holds one row per template field, and `import_batches.mapping_snapshot` stores the exact mapping used, as immutable JSONB. Editing a template creates a new template version row (see Section C) rather than mutating mappings in place.
`import_errors`	**Keep, with a constraint on its role.** Row-level validation outcomes live on `import_rows` and `statement_lines`. `import_errors` is a normalised, queryable record of every error and warning (batch, row, field, code, message) so support and UI queries do not scan JSONB. A database rule must keep the two consistent: an error row always references the row and field it describes.
`user_sessions`	**Keep, narrowly.** Store only a hash of the refresh credential, expiry, revocation and device label. Never store a raw token, password or one-time code. If the chosen identity provider issues and revokes sessions entirely, this table may shrink to a local revocation list - see OD-03 in Section J.
`workspace_settings`	**Keep as a separate one-to-one table.** Tolerance in poisha, default locale and statement completeness expectations are typed, validated financial controls. Burying them in a JSON column on `merchants` would remove CHECK constraints exactly where they matter most.
`merchants`	**Rename in prose, keep the name in SQL.** In product language this is the workspace or seller business. The table name `merchants` is retained to match the approved baseline and because `merchant_id` is the tenant key everywhere.

A.5 Normalisation and relationship problems in the draft

- **Reported values do not belong on `shipments`.** A shipment can appear on more than one statement (an original and a correction). Reported collection, fees and payout are properties of a statement line, not of the shipment. Storing them on the shipment is a repeating-group and overwrite problem.
- **Expected values do not belong on `orders` alone.** Original order COD and expected collectible COD are different amounts under V2.1: delivered shipments carry the amount to collect, documented partial deliveries carry only the documented partial amount, confirmed returns and cancellations carry verified 0, and unknown or in-transit outcomes carry NULL. Both amounts are therefore stored, with the outcome and its evidence, on the shipment and order as specified in Section C.

- **Findings must not be columns on results.** A result has a list of independent findings. Boolean columns such as `has_fee_issue` would invite netting and would lose amounts. Findings are rows.
- **Every tenant table needs its own `merchant_id`.** Deriving tenancy only through a parent join (for example, a finding reaching its merchant through result, then run) makes RLS policies slow, fragile and easy to bypass with a hand-written query. Denormalising `merchant_id` onto every tenant row, protected by composite foreign keys, is deliberate and is not a normalisation violation: the composite keys guarantee the copies agree.
- **Template and rule versioning must be uniform.** Both statement templates and rule versions are immutable once used. The same pattern - version number, validity window, content hash, superseding row - is used for both, so reviewers learn one pattern.
- **No polymorphic foreign keys.** Columns such as `entity_type` plus `entity_id` cannot be enforced by the database. Where the draft implied them (audit logs, evidence, errors), Section C either uses a real foreign key, or accepts an unenforced reference only in `audit_logs` and says so explicitly, with a compensating rule.

A.6 Principal risks

#	Risk	Mitigation in this design
R1	RLS silently not applied: table owner or superuser connection, a policy missing on one table, or a pooled connection carrying a previous tenant's context.	FORCE RLS on every tenant table, a non-owner non-superuser runtime role, transaction-local context set on every transaction, fail-closed policies when context is missing, and the leakage tests in Section I.
R2	Cross-tenant relationships created through ordinary single-column foreign keys (a shipment in tenant A pointing at an order in tenant B).	Composite foreign keys on `(merchant_id, parent_id)` against a `UNIQUE (merchant_id, id)` pair on every parent table.
R3	Missing money silently becoming zero during CSV parsing, defaulting or aggregation (`COALESCE` in a report query).	Per-field data-quality state stored beside every parsed amount, no database DEFAULT of 0 on any money column, CHECK rules tying state to nullability, and the NULL-versus-zero tests in Section I.
R4	A later rule or template edit rewriting history.	Immutable versions, run snapshots of inputs, rules and mappings, and runtime-role privileges that deny UPDATE and DELETE on runs, results, findings and fee lines.
R5	An unconfirmed contractual delivery fee (`D_c`) being assumed from expected or reported fees for a `cod_minus_delivery_fee` base.	`D_c` exists only on a rule version with an explicit confirmation flag and a non-empty evidence reference; otherwise the affected fees and payouts are NULL and the result is `needs_review` with `DQ_MISSING_CONTRACT`.
R6	Bank variance reported before the money picture is complete.	Bank variance is a stored NULL until the receipt is explicitly marked complete and the settlement batch is fully allocated; partial receipts are a status, never a discrepancy.
R7	Duplicate tracking IDs and repeated statement lines being merged or summed.	No uniqueness is claimed on tracking ID alone within a merchant and courier where duplicates legitimately occur; duplicates are preserved, flagged and quarantined for human confirmation, and never summed or spread pro-rata.
R8	Over-engineering for scale that has not been validated.	Shared schema, no partitioning, no sharding, no separate read replicas in M1. Partitioning of `import_rows` and `audit_logs` is noted only as a future option in Section F.

A.7 Major design decisions

Decision	Status	Rationale
Shared database, shared schema, RLS tenancy	CONFIRMED	Approved baseline. Cheapest model that still gives database-enforced isolation, and proportionate for SME volumes.
`merchant_id` on every tenant-owned table, with composite tenant foreign keys	CONFIRMED	Makes every RLS policy a single-column comparison and makes cross-tenant references impossible at the constraint level.
Money as BIGINT poisha; per-field data-quality state beside every source amount	CONFIRMED	V2.2.1 requires absent column, blank row, invalid value and explicit zero to remain distinct forever, not just at parse time.

Runs snapshot inputs, rule versions, template and mapping, and tolerance	CONFIRMED	A historical result must be explainable after rules, templates and settings change.
Findings as independent rows; three separate statuses on each result	CONFIRMED	P0 patch: contractual payout status, data-quality status and overall status (`matched`, `open`, `mismatch`, `needs_review`, `matched_with_tolerance`) are stored separately so no status hides another finding.
Statement lines as a first-class entity between import rows and results	RECOMMENDED	See A.3. Blocks implementation if rejected, because results need a stable, snapshot-able input.
Application-generated UUIDs; no database sequences exposed as identifiers	RECOMMENDED	Avoids leaking tenant volumes through sequential IDs and lets the application create related rows in one transaction.
Soft state changes instead of hard deletes for financial history	RECOMMENDED	Corrections create new records or runs. Hard delete is reserved for the workspace-erasure workflow in Section G, which is a policy decision, not a routine operation.
One active COD-bearing shipment per order, enforced by a partial unique index	CONFIRMED	MVP scope restriction. Split COD is out of scope and must be rejected, not silently accommodated.

A.8 Assumptions - not confirmed requirements

- **OPEN** Sellers work in a single workspace each in M1, while the membership model already supports many users per workspace and one user in many workspaces. Nothing in the schema prevents either.
- **OPEN** CSV is the only ingestion route in M1 for all five couriers. No courier API table is designed, because no courier API contract has been verified.
- **OPEN** Uploaded files are held in object storage outside PostgreSQL, referenced by `file_objects.storage_key`. Storing file bytes in the database is rejected for M1 (OD-05).
- **OPEN** Bank evidence in M1 is a manually confirmed receipt record, as in V2. Bank CSV import is excluded from the MVP and no bank statement table is designed.
- **OPEN** Statement templates are maintained per courier and versioned; whether a merchant may override a shared template is OD-07.

A.9 Conflicts identified, not silently resolved

> **Conflict 1 - audit completeness versus the ban on storing bank account numbers.** Audit and receipt records must be traceable, but no bank account number, in full or in a reversible form, may be stored. Resolution proposed for review: receipts carry a merchant-supplied reference label and the last four digits only if a policy decision explicitly permits even that; the default in Section C stores no digits at all, only a free-text label the merchant controls. This needs the OD-09 decision before implementation.

> **Conflict 2 - workspace erasure versus financial history retention.** A merchant may ask for their workspace to be deleted, while reconciliation history is the product's evidence trail. Section G separates the two: personal data is erased, financial records are anonymised and retained only if a legal and policy decision says they must be. No retention period is asserted in this document.

B. Detailed Entity Relationship Diagram

The diagram is split into four sub-diagrams by module so that it stays readable. Together they cover every table specified in Section C. Notation: `PK` primary key, `FK` foreign key, `UK` part of a unique constraint. Cardinality uses Mermaid crow's-foot notation: `||--o{` is one-to-many with a mandatory parent, `|o--o{` allows a parent with no children yet, and `||--||` is one-to-one. Every tenant-owned child also carries `merchant_id` and is bound to its parent by a composite foreign key on `(merchant_id, parent_id)`; for readability the ERD shows the logical foreign key once.

B.1 Identity, workspace and courier configuration

```
erDiagram
  users ||--o{ merchant_memberships : "is member via"
  merchants ||--o{ merchant_memberships : "has members"
  users ||--o{ user_sessions : "holds"
  merchants ||--|| workspace_settings : "configured by"
  merchants ||--o{ audit_logs : "owns"
  couriers ||--o{ merchant_courier_accounts : "connected as"
  merchants ||--o{ merchant_courier_accounts : "connects"
  couriers ||--o{ courier_statement_templates : "described by"
  courier_statement_templates ||--o{ column_mappings : "maps fields via"
  merchant_courier_accounts ||--o{ courier_rule_versions : "governed by"
  courier_rule_versions ||--o{ courier_rule_evidence : "supported by"
  users {
    uuid id PK
    text email UK
    text display_name
    timestamptz created_at
  }
  merchants {
    uuid id PK
    text name
    text status
    timestamptz created_at
  }
  merchant_memberships {
    uuid id PK
    uuid merchant_id FK
    uuid user_id FK
    text role
    text status
  }
  user_sessions {
    uuid id PK
    uuid user_id FK
    text refresh_token_hash UK
    timestamptz expires_at
  }
  workspace_settings {
    uuid id PK
    uuid merchant_id FK,UK
    bigint tolerance_poisha
    text locale
  }
  couriers {
    uuid id PK
    text code UK
    text name
  }
  merchant_courier_accounts {
    uuid id PK
    uuid merchant_id FK
    uuid courier_id FK
    text merchant_account_ref
    text status
  }
  courier_statement_templates {
    uuid id PK
    uuid courier_id FK
    int version_no
    text content_hash UK
    text status
  }
  column_mappings {
    uuid id PK
    uuid template_id FK
    text canonical_field
    text source_header
  }
  courier_rule_versions {
    uuid id PK
    uuid merchant_id FK
    uuid merchant_courier_account_id FK
    int version_no
    text content_hash
```

```
    boolean dc_confirmed
    text status
  }
  courier_rule_evidence {
    uuid id PK
    uuid merchant_id FK
    uuid rule_version_id FK
    text evidence_type
    text evidence_ref
  }
  audit_logs {
    uuid id PK
    uuid merchant_id FK
    uuid actor_user_id FK
    text action
    timestamptz created_at
  }
```

B.2 Orders, shipments, import and statement lines

```
erDiagram
  merchants ||--o{ orders : "owns"
  orders ||--o{ shipments : "fulfilled by"
  merchants ||--o{ file_objects : "uploads"
  file_objects ||--o{ import_batches : "imported as"
  merchant_courier_accounts ||--o{ import_batches : "supplies statements to"
  courier_statement_templates ||--o{ import_batches : "parsed with"
  import_batches ||--o{ import_rows : "contains"
  import_batches ||--o{ import_errors : "reports"
  import_rows ||--o{ import_errors : "described by"
  import_rows ||--o| statement_lines : "normalised into"
  shipments ||--o{ statement_lines : "reported on"
  shipments ||--o{ shipment_match_candidates : "matched via"
  statement_lines ||--o{ shipment_match_candidates : "candidate for"
  orders {
    uuid id PK
    uuid merchant_id FK
    text merchant_order_ref
    bigint order_cod_poisha
    text currency
  }
  shipments {
    uuid id PK
    uuid merchant_id FK
    uuid order_id FK
    uuid merchant_courier_account_id FK
    text tracking_id_normalised
    text outcome
    bigint expected_collectible_cod_poisha
    boolean is_cod_bearing
    boolean is_superseded
  }
  file_objects {
    uuid id PK
    uuid merchant_id FK
    text storage_key UK
    text sha256 UK
    bigint size_bytes
  }
  import_batches {
    uuid id PK
    uuid merchant_id FK
    uuid file_object_id FK
    uuid template_id FK
    jsonb mapping_snapshot
    text status
    text idempotency_key UK
  }
  import_rows {
    uuid id PK
    uuid merchant_id FK
    uuid import_batch_id FK
    int row_number
    jsonb raw_values
    text parse_status
```

```
  }
  import_errors {
    uuid id PK
    uuid merchant_id FK
    uuid import_batch_id FK
    uuid import_row_id FK
    text field_name
    text error_code
  }
  statement_lines {
    uuid id PK
    uuid merchant_id FK
    uuid import_row_id FK
    uuid shipment_id FK
    bigint reported_collected_poisha
    text reported_collected_state
    bigint reported_payout_poisha
    bigint reported_delivery_fee_poisha
    text reported_delivery_fee_state
  }
  shipment_match_candidates {
    uuid id PK
    uuid merchant_id FK
    uuid statement_line_id FK
    uuid shipment_id FK
    text match_basis
    text decision
  }
```

B.3 Reconciliation

```
erDiagram
  merchants ||--o{ reconciliation_runs : "owns"
  import_batches ||--o{ reconciliation_runs : "reconciled in"
  courier_rule_versions ||--o{ reconciliation_runs : "applied in"
  reconciliation_runs ||--o{ reconciliation_results : "produces"
  shipments ||--o{ reconciliation_results : "result for"
  statement_lines ||--o{ reconciliation_results : "compared in"
  reconciliation_results ||--o{ reconciliation_fee_lines : "broken down by"
  reconciliation_results ||--o{ reconciliation_findings : "raises"
  reconciliation_runs {
    uuid id PK
    uuid merchant_id FK
    uuid import_batch_id FK
    jsonb input_snapshot
    jsonb rule_snapshot
    bigint tolerance_poisha
    text status
    text run_hash UK
  }
  reconciliation_results {
    uuid id PK
    uuid merchant_id FK
    uuid run_id FK
    uuid shipment_id FK
    bigint expected_collectible_poisha
    bigint reported_collected_poisha
    bigint c_poisha
    bigint v_cond_poisha
    bigint v_total_poisha
    text contractual_payout_status
    text data_quality_status
    text overall_status
  }
  reconciliation_fee_lines {
    uuid id PK
    uuid merchant_id FK
    uuid result_id FK
    text fee_code
    bigint expected_basis_poisha
    bigint reported_basis_poisha
    bigint reported_actual_poisha
    text data_quality_state
  }
  reconciliation_findings {
```

```
    uuid id PK
    uuid merchant_id FK
    uuid result_id FK
    text finding_type
    text category
    bigint amount_poisha
    text finding_code
  }
```

B.4 Settlement and supporting records

```
erDiagram
  merchants ||--o{ settlement_batches : "owns"
  merchant_courier_accounts ||--o{ settlement_batches : "settled for"
  reconciliation_runs ||--o{ settlement_batches : "expected from"
  settlement_batches ||--o{ bank_receipts : "paid by"
  settlement_batches ||--o{ settlement_allocations : "allocated via"
  bank_receipts ||--o{ settlement_allocations : "allocated from"
  merchants ||--o{ idempotency_keys : "issues"
  settlement_batches {
    uuid id PK
    uuid merchant_id FK
    uuid merchant_courier_account_id FK
    bigint expected_batch_payout_poisha
    bigint bank_variance_poisha
    text allocation_status
  }
  bank_receipts {
    uuid id PK
    uuid merchant_id FK
    uuid settlement_batch_id FK
    bigint amount_poisha
    boolean is_complete
    text receipt_ref_label
    date received_on
  }
  settlement_allocations {
    uuid id PK
    uuid merchant_id FK
    uuid settlement_batch_id FK
    uuid bank_receipt_id FK
    bigint allocated_poisha
  }
  idempotency_keys {
    uuid id PK
    uuid merchant_id FK
    text key UK
    text operation
    text result_ref
  }
```

B.5 Relationship notes a reviewer should check

- **Mandatory versus optional.** A result must have exactly one run, one shipment and, where a statement line was matched, one statement line; an unmatched or missing-line result is represented with a NULL statement line plus a data-quality finding, never with a fabricated line. A settlement batch may exist with no receipts; a receipt may exist with no allocations; bank variance stays NULL in both cases.
- **Optional rule evidence.** A rule version may exist in draft with no evidence, but it cannot become active for a `cod_minus_delivery_fee` base without a confirmation flag and evidence - enforced by CHECK constraints in Section C, not only by application code.
- **Superseded shipments** keep all relationships and history. They are excluded from active reconciliation by a partial index and query rule, not by deletion or by breaking their foreign keys.
- **Uniqueness that is deliberately absent.** There is no unique constraint on `(merchant_id, courier, tracking_id)`. Duplicate tracking IDs occur in real statements and must be preserved and flagged. Uniqueness is enforced instead on the active COD-bearing shipment per order, on idempotency keys, on file hashes per merchant, and on version numbers within their parent.

C. Table-by-Table Specification

Reading note: every table lists its purpose, then its columns. **Key / constraints** combines primary key (PK), foreign key (FK), unique (UK), CHECK and DEFAULT information to keep the specification readable. **Tenant class** is one of *Global* (shared reference data), *Tenant-owned* (carries `merchant_id`, RLS enforced), or *Identity* (user-level, protected by membership checks rather than a merchant RLS policy). Indexes listed are in addition to the primary key. A composite `UNIQUE (merchant_id, id)` is required on every tenant-owned parent table so children can use composite tenant foreign keys; it is stated once here and applies throughout.

C.1 Identity and workspace

users - a person who can sign in. Identity class. No RLS merchant policy; access controlled by authentication and membership.

Column	Type	Null	Key / constraints
`id`	UUID	NOT NULL	PK
`email`	TEXT	NOT NULL	UK, stored lower-cased; CHECK contains '@'
`display_name`	TEXT	NULL	-
`password_hash`	TEXT	NULL	NULL when an external identity provider is used (OD-03). Never store a plain password.
`status`	TEXT	NOT NULL	CHECK in ('active','invited','suspended','closed'); DEFAULT 'active'
`created_at / updated_at`	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Mutable, audited. Indexes: unique on lower(email) is the lookup index.

merchants - the tenant workspace (a seller business). Tenant root. RLS: a member may read only workspaces they belong to; creation is a controlled function.

Column	Type	Null	Key / constraints
`id`	UUID	NOT NULL	PK. This value is `merchant_id` on every tenant table.
`name`	TEXT	NOT NULL	CHECK length 1-200
`trading_name`	TEXT	NULL	-
`status`	TEXT	NOT NULL	CHECK in ('active','suspended','closing','closed'); DEFAULT 'active'
`created_by_user_id`	UUID	NOT NULL	FK to users(id)
`created_at / updated_at`	TIMESTAMPTZ	NOT NULL	DEFAULT now()
`closed_at`	TIMESTAMPTZ	NULL	Set only by the workspace-closure workflow in Section G.

Immutability: Mutable except identity fields after closure. No tenant column of its own.

merchant_memberships - which users belong to which workspace, and in what role. Tenant-owned. RLS enforced.

Column	Type	Null	Key / constraints
`id`	UUID	NOT NULL	PK
`merchant_id`	UUID	NOT NULL	FK to merchants(id); UK part of (merchant_id, user_id)
`user_id`	UUID	NOT NULL	FK to users(id)

role	TEXT	NOT NULL	CHECK in ('owner','admin','member','viewer')
status	TEXT	NOT NULL	CHECK in ('active','invited','revoked'); DEFAULT 'invited'
invited_by_user_id	UUID	NULL	FK to users(id)
created_at / updated_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Mutable, audited; role changes are audit-logged. Indexes: (merchant_id, status), (user_id, status). Partial unique index guaranteeing at least the possibility of one active owner is an application rule, not a database constraint - the database cannot express "at least one" without a deferred trigger, which is noted as OD-11.

user_sessions - revocable sign-in sessions. Identity class.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
user_id	UUID	NOT NULL	FK to users(id)
refresh_token_hash	TEXT	NOT NULL	UK. Hash only; a raw token must never be stored.
device_label	TEXT	NULL	Merchant/user-supplied label only.
expires_at	TIMESTAMPTZ	NOT NULL	-
revoked_at	TIMESTAMPTZ	NULL	-
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Mutable for revocation only. Indexes: (user_id, expires_at). Expired and revoked rows are purged under Section G.

C.2 Courier configuration

couriers - global reference list of supported couriers. Global class. No merchant RLS; read-only for the runtime role.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
code	TEXT	NOT NULL	UK; CHECK in ('steadfast','pathao','paperfly','redx','ecourier'). Adding a code is a migration, not user data.
name	TEXT	NOT NULL	Display name only. No rate, fee or format information is stored on this table.
is_active	BOOLEAN	NOT NULL	DEFAULT true

Immutability: Mutable by migration/admin only. This table must never acquire rate columns: rates are per merchant, per account and per evidence-backed rule version.

merchant_courier_accounts - a merchant's account with one courier. Tenant-owned. RLS enforced.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
courier_id	UUID	NOT NULL	FK to couriers(id)

merchant_account_ref	TEXT	NULL	The merchant's own account identifier as they supply it. Not a credential.
display_label	TEXT	NULL	-
status	TEXT	NOT NULL	CHECK in ('active','inactive'); DEFAULT 'active'
created_at / updated_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Mutable, audited. Indexes: (merchant_id, courier_id, status). No unique rule prevents two accounts with the same courier - a seller may legitimately hold two - so duplicates are allowed and labelled.

courier_statement_templates - a versioned description of one courier statement layout. Global template with optional merchant ownership (OD-07); the columns below show the global form. Versioned and immutable once used.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
courier_id	UUID	NOT NULL	FK to couriers(id); UK part of (courier_id, version_no)
version_no	INTEGER	NOT NULL	CHECK > 0
name	TEXT	NOT NULL	-
header_fingerprint	TEXT	NULL	Normalised header signature used only to suggest a template, never to auto-apply financial rules.
content_hash	TEXT	NOT NULL	Hash of the template plus its column mappings.
status	TEXT	NOT NULL	CHECK in ('draft','active','retired'); DEFAULT 'draft'
evidence_ref	TEXT	NULL	Where the layout was verified from. A template layout is not contract evidence and must not be used as such.
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Versioned. Once any import batch references a template, its mappings may not be edited; a correction is a new version.

column_mappings - one canonical field mapped from one source header, within one template. Child of a template.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
template_id	UUID	NOT NULL	FK to courier_statement_templates(id); UK part of (template_id, canonical_field)
canonical_field	TEXT	NOT NULL	CHECK in the canonical field list: tracking_id, merchant_order_ref, reported_collected, reported_payout, delivery_fee, cod_fee, return_fee, other_fee, shipment_status, statement_date and the remaining fields enumerated in the implementation ticket. The list is closed.
source_header	TEXT	NOT NULL	Header text exactly as it appears in the source file.
source_header_normalised	TEXT	NOT NULL	Lower-cased, trimmed form used for matching.
is_required	BOOLEAN	NOT NULL	DEFAULT false. A missing required column makes the whole column absent-state for every row (Section D).

Immutability: Versioned with its template. Indexes: (template_id). Per-import truth is the snapshot on import_batches, not this table.

courier_rule_versions - one immutable, evidence-backed version of the fee and payout rules for one merchant courier account. Tenant-owned. RLS enforced. Versioned and immutable once active.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
merchant_courier_account_id	UUID	NOT NULL	Composite FK (merchant_id, merchant_courier_account_id) to merchant_courier_accounts; UK part of (account, version_no)
version_no	INTEGER	NOT NULL	CHECK > 0
status	TEXT	NOT NULL	CHECK in ('draft','active','superseded','retired'); DEFAULT 'draft'
effective_from / effective_to	TIMESTAMPTZ	NULL / NULL	CHECK effective_to is NULL or > effective_from. Overlap of active versions for one account is prevented by an exclusion rule or a controlled activation transaction (OD-10).
cod_fee_base	TEXT	NULL	CHECK in ('expected_collectible','reported_collected','cod_minus_delivery_fee') or NULL when unknown. NULL means the base is not established and no contractual COD fee may be calculated.
cod_fee_rate_bps	INTEGER	NULL	Rate in basis points, not money. CHECK between 0 and 10000 when present.
contractual_delivery_fee_poisha (D_c)	BIGINT	NULL	The merchant-confirmed contractual delivery fee used only as the subtractor for a cod_minus_delivery_fee base. Never copied from expected or reported delivery fees.
dc_confirmed	BOOLEAN	NOT NULL	DEFAULT false. Must be explicitly set; supplying a D_c value alone never sets it.
dc_evidence_ref	TEXT	NULL	Required, non-empty, when dc_confirmed is true.
delivery_fee_poisha, return_fee_poisha	BIGINT	NULL	Contractual fee amounts only where merchant-confirmed. NULL when unknown.
vat_mode	TEXT	NULL	CHECK in ('included_rate_on_net','included_rate_on_gross','excluded','unknown') or NULL. An included-tax breakdown is calculated only when mode and rate are both known.
vat_rate_bps	INTEGER	NULL	CHECK between 0 and 10000 when present.
rule_definition	JSONB	NOT NULL	Full structured rule definition for fee components not given their own columns. Validated against a versioned schema in the application and hashed.
content_hash	TEXT	NOT NULL	Hash of the complete rule definition, copied into every run snapshot.
created_by_user_id	UUID	NOT NULL	FK to users(id)
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Key CHECK (paraphrased; full form in the migration review): if cod_fee_base = 'cod_minus_delivery_fee' and the version is active, then dc_confirmed must be true, dc_evidence_ref must be non-empty and contractual_delivery_fee_poisha must be NOT NULL. If any of those fails, the version may exist as a draft but any reconciliation using it must produce NULL fees and payouts for the affected base and a DQ_MISSING_CONTRACT finding. Indexes: (merchant_id, merchant_courier_account_id, status), unique partial index allowing only one active version per account at a time where the activation model requires it (OD-10).

courier_rule_evidence - provenance for a rule version: what the merchant confirmed, and how. Tenant-owned. RLS enforced. Append-only.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
merchant_id	UUID	NOT NULL	FK to merchants(id)

rule_version_id	UUID	NOT NULL	Composite FK (merchant_id, rule_version_id) to courier_rule_versions
evidence_type	TEXT	NOT NULL	CHECK in ('merchant_confirmation','signed_rate_card','courier_letter','statement_annotation','other_documented')
evidence_ref	TEXT	NOT NULL	CHECK length > 0. Reference to the merchant-held document or confirmation record. Public pricing pages are not acceptable evidence and must be rejected by application validation.
file_object_id	UUID	NULL	Composite FK to file_objects when the evidence is an uploaded file.
confirmed_by_user_id	UUID	NOT NULL	FK to users(id)
confirmed_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()
note	TEXT	NULL	-

Immutability: Append-only. Correcting evidence adds a new row and, where the correction changes a rule, a new rule version. Indexes: (merchant_id, rule_version_id).

C.3 Orders and shipments

orders - a seller order as the merchant records it. Tenant-owned. RLS enforced.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
merchant_order_ref	TEXT	NOT NULL	The merchant's own order identifier. UK part of (merchant_id, merchant_order_ref) only where the merchant confirms refs are unique; otherwise indexed, not unique (OD-08).
order_cod_poisha	BIGINT	NULL	Original order COD. NULL when unknown - never defaulted to 0 and never copied onto shipments as an assumption. CHECK >= 0 when present.
currency	TEXT	NOT NULL	CHECK = 'BDT' in M1; DEFAULT 'BDT'. Multi-currency is out of scope.
ordered_at	TIMESTAMPTZ	NULL	-
source	TEXT	NOT NULL	CHECK in ('manual','csv_import','demo_synthetic'); DEFAULT 'manual'
created_at / updated_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Mutable before reconciliation; once an order's shipments appear in a run, the run holds the snapshot and later edits never alter the run. Indexes: (merchant_id, merchant_order_ref), (merchant_id, created_at).

shipments - one physical shipment under an order, with its confirmed outcome and expected collectible COD. Tenant-owned. RLS enforced.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
order_id	UUID	NOT NULL	Composite FK (merchant_id, order_id) to orders
merchant_courier_account_id	UUID	NULL	Composite FK to merchant_courier_accounts. NULL until the courier account is known.
tracking_id_raw	TEXT	NULL	Exactly as supplied. Never overwritten by the normalised form.
tracking_id_normalised	TEXT	NULL	Trimmed, case-normalised form used for strong matching. Blank after normalisation is stored as NULL plus a data-quality flag, not as an empty string.

outcome	TEXT	NOT NULL	CHECK in ('delivered','partially_delivered','returned','cancelled','in_transit','unknown'); DEFAULT 'unknown'
outcome_source	TEXT	NULL	CHECK in ('merchant_confirmed','statement','system_inferred_label_only') - an inferred label may never set a financial amount.
expected_collectible_cod_poisha (E basis)	BIGINT	NULL	Per V2.1: delivered = amount to collect; partially delivered = documented partial amount only, else NULL; confirmed return or cancellation = verified 0; unknown or in-transit = NULL. CHECK >= 0 when present.
partial_amount_evidence_ref	TEXT	NULL	Required non-empty when a partial amount is recorded from documentation.
is_cod_bearing	BOOLEAN	NOT NULL	DEFAULT false
is_superseded	BOOLEAN	NOT NULL	DEFAULT false. Superseded rows remain queryable history.
superseded_by_shipment_id	UUID	NULL	Composite FK to shipments (same merchant, same order). CHECK: set only when is_superseded is true.
superseded_reason	TEXT	NULL	-
shipped_on	DATE	NULL	Used by the weak-match date window only when both dates exist.
created_at / updated_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Mutable, with outcome changes audited; runs snapshot what they used. Indexes: (merchant_id, tracking_id_normalised), (merchant_id, order_id), (merchant_id, merchant_courier_account_id, outcome). **Partial unique index:** `UNIQUE (merchant_id, order_id) WHERE is_cod_bearing AND NOT is_superseded` - this is the database enforcement of "one active COD-bearing shipment per order". Any split-COD attempt fails this index and must be surfaced for review, never accommodated silently.

C.4 CSV import

file_objects - metadata for an uploaded file held outside the database. Tenant-owned. RLS enforced. Append-only.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
storage_key	TEXT	NOT NULL	UK. Opaque object-storage key; never a public URL.
original_filename	TEXT	NOT NULL	For display only; never trusted as a path.
sha256	TEXT	NOT NULL	UK part of (merchant_id, sha256) - re-uploading the identical file for the same merchant is detected, not silently duplicated.
size_bytes	BIGINT	NOT NULL	CHECK >= 0
content_type	TEXT	NOT NULL	CHECK in ('text/csv','application/vnd.ms-excel') for M1 CSV scope.
uploaded_by_user_id	UUID	NOT NULL	FK to users(id)
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()
purged_at	TIMESTAMPTZ	NULL	Set when the stored bytes are deleted under Section G; the metadata row and hash remain.

import_batches - one CSV import attempt for one courier account. Tenant-owned. RLS enforced.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)

merchant_id	UUID	NOT NULL	FK to merchants(id)
file_object_id	UUID	NOT NULL	Composite FK to file_objects
merchant_courier_account_id	UUID	NOT NULL	Composite FK to merchant_courier_accounts
template_id	UUID	NULL	FK to courier_statement_templates. NULL while unmapped.
template_version_no	INTEGER	NULL	Copied at mapping time so the batch is self-describing.
mapping_snapshot	JSONB	NULL	The exact mapping applied. Immutable once status leaves 'uploaded'. This snapshot, not the live template, is what a run records.
status	TEXT	NOT NULL	CHECK in ('uploaded','mapped','parsed','parse_failed','ready','superseded'); DEFAULT 'uploaded'
row_count / parsed_row_count / error_row_count	INTEGER	NOT NULL	DEFAULT 0; CHECK >= 0. Counts are operational metadata, never money, so a 0 default is correct here.
statement_period_start / _end	DATE	NULL	Merchant-declared or parsed period, labelled by source in the application.
idempotency_key	TEXT	NULL	UK part of (merchant_id, idempotency_key) where present.
created_by_user_id	UUID	NOT NULL	FK to users(id)
created_at / updated_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: status and counts are mutable during import; `mapping_snapshot`, file and account references are immutable after parsing. Indexes: (merchant_id, status, created_at), (merchant_id, merchant_courier_account_id, created_at).

import_rows - one raw CSV row, preserved verbatim. Tenant-owned. RLS enforced. Append-only after parsing.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
import_batch_id	UUID	NOT NULL	Composite FK to import_batches; UK part of (import_batch_id, row_number)
row_number	INTEGER	NOT NULL	CHECK > 0. Position in the source file, counting data rows as defined in the import specification.
raw_values	JSONB	NOT NULL	Object of source header to raw string exactly as read, including blanks as empty strings and absent columns as absent keys. This distinction is load-bearing: absent key, blank value and "0" are three different states.
raw_line_hash	TEXT	NOT NULL	Hash of the raw row, used for duplicate-row detection.
parse_status	TEXT	NOT NULL	CHECK in ('parsed','parsed_with_warnings','failed','skipped_blank')
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Indexes: (merchant_id, import_batch_id, row_number). Duplicate rows are kept and flagged; no unique constraint merges them.

import_errors - one error or warning against a batch, row or field. Tenant-owned. RLS enforced. Append-only.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
merchant_id	UUID	NOT NULL	FK to merchants(id)

import_batch_id	UUID	NOT NULL	Composite FK to import_batches
import_row_id	UUID	NULL	Composite FK to import_rows. NULL for batch-level errors such as a missing column.
field_name	TEXT	NULL	Canonical field name, or NULL for row/batch-level errors.
severity	TEXT	NOT NULL	CHECK in ('error','warning')
error_code	TEXT	NOT NULL	Closed list including COLUMN_ABSENT, VALUE_BLANK, VALUE_INVALID, VALUE_UNPARSABLE, DUPLICATE_TRACKING, AMBIGUOUS_MATCH. Codes are stable API surface.
message	TEXT	NOT NULL	Human-readable explanation in English; Bangla explanation is supplied by the presentation layer for financial statuses.
raw_value	TEXT	NULL	The offending raw string, preserved for diagnosis. NULL when the problem is absence itself.
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Indexes: (merchant_id, import_batch_id, severity), (merchant_id, import_row_id).

statement_lines - the normalised courier-reported values for one imported row, with per-field data-quality state. Tenant-owned. RLS enforced. Append-only; a corrected statement creates new lines in a new batch.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
import_row_id	UUID	NOT NULL	Composite FK to import_rows; UK - one normalised line per raw row
import_batch_id	UUID	NOT NULL	Composite FK to import_batches (denormalised for tenant-scoped queries, kept consistent by the composite key through the row)
shipment_id	UUID	NULL	Composite FK to shipments. NULL while unmatched; set only by a strong match or a human-confirmed candidate, never by amount alone.
match_status	TEXT	NOT NULL	CHECK in ('strong_matched','human_confirmed','unmatched','ambiguous','duplicate'); DEFAULT 'unmatched'
tracking_id_raw / _normalised	TEXT	NULL	As reported on this statement, preserved separately from the shipment's own values.
merchant_order_ref_raw	TEXT	NULL	As reported; used only by the defined weak-match fallback.
reported_status_raw	TEXT	NULL	Courier status text exactly as reported.
statement_date	DATE	NULL	-
reported_collected_poisha (R)	BIGINT	NULL	CHECK >= 0 when present. No DEFAULT.
reported_collected_state	TEXT	NOT NULL	CHECK in ('present_valid','explicit_zero','column_absent','blank','invalid'). See the state rules below.
reported_payout_poisha (P_r)	BIGINT	NULL	CHECK >= 0 when present, except that a statement may legitimately report a negative net amount for a return-heavy line - that case is OD-12 and must not be silently clamped to 0.
reported_payout_state	TEXT	NOT NULL	Same CHECK list as reported_collected_state.
reported_delivery_fee_poisha (D_r)	BIGINT	NULL	Reported delivery fee. Kept strictly separate from expected D_e and contractual D_c.
reported_delivery_fee_state	TEXT	NOT NULL	Same CHECK list.
reported_cod_fee_poisha, reported_return_fee_poisha, reported_other_fee_poisha	BIGINT	NULL	Each with a companion _state column using the same CHECK list. Additional courier fee columns go to fee detail only through a template-defined canonical field, never into an untyped catch-all amount.

`created_at`	TIMESTAMPTZ	NOT NULL	DEFAULT now()

> **State-to-value CHECK, applied to every amount/state pair.** `present_valid` requires a NOT NULL value greater than 0; `explicit_zero` requires the value to be exactly 0; `column_absent`, `blank` and `invalid` all require the value to be NULL. This single pattern is what makes "missing is not zero" a database guarantee rather than a parsing convention. It also drives Section D: any fee line in an absent, blank or invalid state makes the affected totals NULL, not smaller.

Indexes: (merchant_id, import_batch_id), (merchant_id, shipment_id), (merchant_id, tracking_id_normalised), (merchant_id, match_status) partial on unmatched and ambiguous.

shipment_match_candidates - every candidate match considered for a statement line, with its basis and decision. Tenant-owned. RLS enforced. Append-only, with a single decision update recorded by audit.

Column	Type	Null	Key / constraints
`id`	UUID	NOT NULL	PK
`merchant_id`	UUID	NOT NULL	FK to merchants(id)
`statement_line_id`	UUID	NOT NULL	Composite FK to statement_lines
`shipment_id`	UUID	NOT NULL	Composite FK to shipments
`match_basis`	TEXT	NOT NULL	CHECK in ('strong_tracking','weak_order_ref'). There is deliberately no 'amount' value in this list: amount alone can never create a candidate.
`rank`	INTEGER	NOT NULL	CHECK > 0. Deterministic ordering defined in Section D.
`decision`	TEXT	NOT NULL	CHECK in ('proposed','confirmed','rejected','superseded'); DEFAULT 'proposed'
`decided_by_user_id`	UUID	NULL	FK to users(id). Required by application rule when decision is confirmed or rejected by a human.
`decided_at`	TIMESTAMPTZ	NULL	-
`created_at`	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Partial unique index: at most one confirmed candidate per statement line - `UNIQUE (statement_line_id) WHERE decision = 'confirmed'`. Indexes: (merchant_id, statement_line_id), (merchant_id, shipment_id).

C.5 Reconciliation

reconciliation_runs - one immutable execution of the reconciliation engine over one import batch. Tenant-owned. RLS enforced. Append-only.

Column	Type	Null	Key / constraints
`id`	UUID	NOT NULL	PK; UK part of (merchant_id, id)
`merchant_id`	UUID	NOT NULL	FK to merchants(id)
`import_batch_id`	UUID	NOT NULL	Composite FK to import_batches
`merchant_courier_account_id`	UUID	NOT NULL	Composite FK to merchant_courier_accounts
`rule_version_id`	UUID	NULL	Composite FK to courier_rule_versions. NULL only where no rule version exists, in which case every contractual output of the run is NULL and every result is needs_review.

rule_version_no / rule_content_hash	INTEGER / TEXT	NULL	Copied from the rule version at run time.
rule_snapshot	JSONB	NOT NULL	Complete rule definition as applied, including D_c, its confirmation flag and evidence reference. Immutable.
input_snapshot	JSONB	NOT NULL	Identifiers and hashes of the statement lines, shipments, mapping snapshot and template version used. Immutable.
mapping_snapshot	JSONB	NOT NULL	Copied from the import batch. Immutable.
tolerance_poisha	BIGINT	NOT NULL	CHECK >= 0. Copied from workspace settings at run time; DEFAULT is not used - the value must be read and recorded. Approved default tolerance is 0 poisha, set in settings, not in this table.
engine_version	TEXT	NOT NULL	Version of the calculation engine that produced the run, so a later engine change never silently reinterprets an old run.
status	TEXT	NOT NULL	CHECK in ('running','completed','failed'); DEFAULT 'running'. A failed run keeps its row and its error; it produces no results.
run_hash	TEXT	NOT NULL	UK part of (merchant_id, run_hash). Hash of inputs, rule snapshot, tolerance and engine version. Identical inputs must produce an identical hash - the determinism test in Section I.
idempotency_key	TEXT	NULL	UK part of (merchant_id, idempotency_key) where present.
created_by_user_id	UUID	NOT NULL	FK to users(id)
created_at / completed_at	TIMESTAMPTZ	NOT NULL / NULL	created_at DEFAULT now(). completed_at is the only post-creation update permitted, together with status.

Immutability: Append-only. The runtime role is granted INSERT and SELECT, plus a narrow UPDATE of status and completed_at only through a column-level grant or a controlled function (Section E). There is no delete path and no API endpoint that edits or deletes a run. Indexes: (merchant_id, import_batch_id, created_at), (merchant_id, created_at).

reconciliation_results - the calculated comparison for one shipment in one run. Tenant-owned. RLS enforced. Append-only.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id); UK part of (run_id, shipment_id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
run_id	UUID	NOT NULL	Composite FK to reconciliation_runs
shipment_id	UUID	NOT NULL	Composite FK to shipments
statement_line_id	UUID	NULL	Composite FK to statement_lines. NULL means no matched line; a data-quality finding must explain it.
expected_collectible_poisha (E)	BIGINT	NULL	Snapshot of the shipment's expected collectible COD at run time.
reported_collected_poisha (R)	BIGINT	NULL	Snapshot from the statement line.
c_poisha (C = R - E)	BIGINT	NULL	NULL when either input is NULL. CHECK by construction in the engine; the database stores the value and the identity checks below.
x_e_poisha (X_e)	BIGINT	NULL	Total expected-basis deductions: the sum of expected-basis fee lines. NULL if any required line is NULL.
x_r_poisha (X_r)	BIGINT	NULL	Total deductions actually reported. NULL if any required reported line is in an absent, blank or invalid state.
x_upper_r_poisha (X_R)	BIGINT	NULL	Total deductions recalculated on the reported basis under the confirmed rules.
delta_poisha (delta = X_R - X_e)	BIGINT	NULL	-

v_r_poisha (V_R = X_r - X_R)	BIGINT	NULL	Reported-basis fee/tax variance.
p_e_poisha (P_e)	BIGINT	NULL	Expected-collection scenario payout.
p_cond_poisha (P_cond)	BIGINT	NULL	Payout conditional on reported collection - the contractual comparison target for a reported_collected base.
p_r_poisha (P_r)	BIGINT	NULL	Reported payout, snapshotted from the statement line.
s_poisha (S)	BIGINT	NULL	Statement inconsistency residual, S = P_r - (R - X_r). Never forced to zero.
v_cond_poisha (V_cond)	BIGINT	NULL	V_cond = P_r - P_cond, and must equal S - V_R when all inputs are present.
v_total_poisha (V_total)	BIGINT	NULL	V_total = P_r - P_e, and must equal C - delta - V_R + S when all inputs are present. Scenario-only for a reported_collected base.
contractual_payout_status	TEXT	NOT NULL	CHECK in ('matched','mismatch','matched_with_tolerance','not_calculable')
data_quality_status	TEXT	NOT NULL	CHECK in ('ok','has_warnings','needs_review')
overall_status	TEXT	NOT NULL	CHECK in ('matched','open','mismatch','needs_review','matched_with_tolerance'). Derivation rules in Section D.
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Identity CHECKs, applied only when every term is NOT NULL: `v_total_poisha = c_poisha - delta_poisha - v_r_poisha + s_poisha` and `v_cond_poisha = s_poisha - v_r_poisha`. Where any term is NULL, the dependent values must also be NULL - a partial identity is never stored. Indexes: (merchant_id, run_id), (merchant_id, shipment_id), (merchant_id, overall_status) partial for open, mismatch and needs_review work queues.

reconciliation_fee_lines - one fee component of one result, in all three bases, with its rounding and data-quality state. Tenant-owned. RLS enforced. Append-only.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (result_id, fee_code)
merchant_id	UUID	NOT NULL	FK to merchants(id)
result_id	UUID	NOT NULL	Composite FK to reconciliation_results
fee_code	TEXT	NOT NULL	CHECK in ('delivery_fee','cod_fee','return_fee','vat_on_fee','other_fee'). The list is closed; a new fee type is a specification change.
expected_basis_poisha	BIGINT	NULL	Fee calculated on the expected basis (contributes to X_e).
reported_basis_poisha	BIGINT	NULL	Fee recalculated on the reported basis (contributes to X_R). For a reported_collected COD fee base this is the contractual fee.
reported_actual_poisha	BIGINT	NULL	Fee as reported on the statement line (contributes to X_r).
included_tax_poisha	BIGINT	NULL	Breakdown only, never subtracted again. NOT NULL only when the rule states an included-tax formula and the rate; otherwise NULL while the gross fee still counts once.
data_quality_state	TEXT	NOT NULL	CHECK in ('present_valid','explicit_zero','column_absent','blank','invalid','rule_unknown','contract_missing')
rounding_note	TEXT	NULL	Records that half-up rounding was applied per line and the unrounded basis where that aids review. No floating-point value is stored.
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Indexes: (merchant_id, result_id). A result's X_e, X_R and X_r must equal the sum of its fee lines in the matching basis, or be NULL when any contributing line is NULL - verified by the test suite, and by a deferred consistency check in the run-completion transaction.

reconciliation_findings - one independent finding raised by one result. Tenant-owned. RLS enforced. Append-only.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (result_id, finding_code)

merchant_id	UUID	NOT NULL	FK to merchants(id)
run_id	UUID	NOT NULL	Composite FK to reconciliation_runs (denormalised for work-queue queries)
result_id	UUID	NOT NULL	Composite FK to reconciliation_results
category	TEXT	NOT NULL	CHECK in ('financial','data_quality')
finding_type	TEXT	NOT NULL	CHECK in ('collection','fee_tax','statement_arithmetic','conditional_payout','scenario','data_quality')
finding_code	TEXT	NOT NULL	Stable codes including UNDER_COLLECTION, OVER_COLLECTION, FEE_VARIANCE, STATEMENT_INCONSISTENT, CONDITIONAL_PAYOUT_VARIANCE, SCENARIO_VARIANCE, DQ_MISSING_CONTRACT, DQ_COLUMN_ABSENT, DQ_VALUE_BLANK, DQ_VALUE_INVALID, DQ_AMBIGUOUS_MATCH, DQ_DUPLICATE_TRACKING, DQ_MISSING_OUTCOME.
amount_poisha	BIGINT	NULL	The gross amount for this finding alone. NULL for data-quality findings with no amount. Findings are never summed into a single loss figure by the database or by a view.
is_within_tolerance	BOOLEAN	NOT NULL	DEFAULT false. Judged against the run's recorded tolerance.
explanation_en	TEXT	NOT NULL	Plain-English explanation with the numbers and their working.
explanation_bn	TEXT	NULL	Short Bangla explanation for financial statuses and money terms, per the approved UI convention.
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Indexes: (merchant_id, run_id, category), (merchant_id, finding_code), partial index on data-quality findings for the review queue. No column on this table, and no view, may label a finding a "confirmed loss": an unexplained difference is stored as a variance with its code, nothing more.

C.6 Settlement

settlement_batches - the payout expected from one courier account for one period, against which bank receipts are checked. Tenant-owned. RLS enforced.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
merchant_courier_account_id	UUID	NOT NULL	Composite FK to merchant_courier_accounts
run_id	UUID	NULL	Composite FK to reconciliation_runs, where the batch derives from a run.
period_start / period_end	DATE	NULL	CHECK period_end is NULL or >= period_start
expected_batch_payout_poisha	BIGINT	NULL	The batch-level amount expected from courier-reported figures. NULL when it cannot be calculated exactly. Kept separate from any bank-confirmed amount.
allocated_total_poisha	BIGINT	NULL	Sum of allocations, maintained in the allocation transaction. NULL until at least one allocation exists.
bank_variance_poisha	BIGINT	NULL	**Must remain NULL** until every linked receipt is explicitly marked complete AND the batch is fully allocated. Enforced by the write path and by a CHECK pattern described in Section D; never defaulted.
allocation_status	TEXT	NOT NULL	CHECK in ('unallocated','partially_allocated','fully_allocated'); DEFAULT 'unallocated'
bank_status	TEXT	NOT NULL	CHECK in ('no_receipt','partial_receipt','receipt_complete_unallocated','fully_reconciled'); DEFAULT 'no_receipt'

created_at / updated_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Mutable for status and allocation totals; expected amounts, once a receipt is allocated against them, change only by creating a corrected batch that supersedes this one. Indexes: (merchant_id, merchant_courier_account_id, period_start), (merchant_id, allocation_status).

bank_receipts - money the merchant confirms was received, recorded as manual bank-confirmation evidence. Tenant-owned. RLS enforced.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (merchant_id, id)
merchant_id	UUID	NOT NULL	FK to merchants(id)
settlement_batch_id	UUID	NULL	Composite FK to settlement_batches. NULL allows a receipt to be recorded before its batch is identified; it then cannot contribute to any variance.
amount_poisha	BIGINT	NOT NULL	CHECK > 0. A receipt is money received; a correction is a new receipt record with a note, not a negative edit.
received_on	DATE	NULL	-
receipt_ref_label	TEXT	NULL	Merchant-supplied label only. **No bank account number column exists on this table, and none may be added.** See Conflict 1 in Section A and OD-09.
is_complete	BOOLEAN	NOT NULL	DEFAULT false. Set only by an explicit merchant action stating this receipt is the complete receipt for its purpose. Never inferred from amounts matching.
marked_complete_by_user_id / _at	UUID / TIMESTAMPTZ	NULL	Required together when is_complete is true.
evidence_note	TEXT	NULL	How the merchant confirmed receipt. Kept separate from courier-reported payout at every level.
created_at / updated_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Multiple partial receipts per batch are expected and supported. A partial receipt is a status, not a discrepancy. Indexes: (merchant_id, settlement_batch_id), (merchant_id, received_on).

settlement_allocations - how much of one receipt is allocated to one settlement batch. Tenant-owned. RLS enforced. Append-only; a reallocation reverses and re-creates.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK; UK part of (bank_receipt_id, settlement_batch_id, created_at) pattern - in practice uniqueness is on the active allocation pair, below
merchant_id	UUID	NOT NULL	FK to merchants(id)
settlement_batch_id	UUID	NOT NULL	Composite FK to settlement_batches
bank_receipt_id	UUID	NOT NULL	Composite FK to bank_receipts
allocated_poisha	BIGINT	NOT NULL	CHECK > 0
is_active	BOOLEAN	NOT NULL	DEFAULT true. Reversal sets a compensating row rather than deleting.
reverses_allocation_id	UUID	NULL	Self-referencing composite FK for reversal rows.
created_by_user_id	UUID	NOT NULL	FK to users(id)
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Rules: the sum of active allocations from one receipt must never exceed that receipt's amount, and the sum into one batch defines its allocated total. PostgreSQL cannot express a cross-row sum as a plain CHECK, so this is enforced in the allocation transaction with row locks, plus a deferred constraint

trigger reviewed as part of implementation - flagged here so no reviewer assumes a CHECK exists. Allocations are never spread pro-rata automatically and never created by the system from an amount match alone. Indexes: (merchant_id, settlement_batch_id), (merchant_id, bank_receipt_id).

C.7 Audit, settings and supporting records

workspace_settings - typed per-workspace controls. Tenant-owned. RLS enforced. One row per merchant.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
merchant_id	UUID	NOT NULL	FK to merchants(id); UK - strictly one-to-one
tolerance_poisha	BIGINT	NOT NULL	CHECK >= 0; DEFAULT 0. Settable only by Owner or Admin. The gross difference is always stored and shown regardless of tolerance.
locale	TEXT	NOT NULL	CHECK in ('en','en_bn'); DEFAULT 'en_bn' - English UI with Bangla explanations for financial statuses.
updated_by_user_id	UUID	NULL	FK to users(id)
created_at / updated_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Mutable, audited. A tolerance change affects future runs only, because each run copies the tolerance it used.

audit_logs - append-only record of consequential actions. Tenant-owned where a merchant context exists. RLS enforced for reads.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
merchant_id	UUID	NULL	FK to merchants(id). NULL only for pre-workspace identity events such as sign-in.
actor_user_id	UUID	NULL	FK to users(id). NULL for system actions, which must then set actor_type.
actor_type	TEXT	NOT NULL	CHECK in ('user','system')
action	TEXT	NOT NULL	Closed vocabulary, for example rule_version.activated, match.confirmed, receipt.marked_complete, tolerance.changed, workspace.closed.
entity_table	TEXT	NULL	Name of the affected table. **Not a foreign key** - this is the one deliberate exception, because audit must survive deletion of the rows it describes. Compensating rule: entity_id is recorded with a snapshot of the identifying fields in detail.
entity_id	UUID	NULL	Unenforced reference; see above.
detail	JSONB	NOT NULL	Before/after values for the changed fields. Must never contain a bank account number, a raw credential or a full uploaded file.
created_at	TIMESTAMPTZ	NOT NULL	DEFAULT now()

Immutability: Append-only. The runtime role has INSERT and SELECT only. Indexes: (merchant_id, created_at), (merchant_id, action, created_at), (actor_user_id, created_at).

idempotency_keys - one stored key per retried operation, scoped to a merchant. Tenant-owned. RLS enforced.

Column	Type	Null	Key / constraints
id	UUID	NOT NULL	PK
merchant_id	UUID	NOT NULL	FK to merchants(id); UK part of (merchant_id, key, operation)
key	TEXT	NOT NULL	Client-supplied key.

`operation`	TEXT	NOT NULL	CHECK in ('import.create','run.create','receipt.create','allocation.create')
`result_ref`	TEXT	NULL	Identifier of the object created by the first successful call, returned on replay.
`request_hash`	TEXT	NOT NULL	A replay with the same key but a different request hash is an error, not a silent success.
`created_at / expires_at`	TIMESTAMPTZ	NOT NULL	Expiry window is OD-13.

D. Financial Data Integrity

D.1 Monetary storage and rounding

All money is `BIGINT` integer poisha. The database stores only integers; no financial calculation in SQL, in the ORM or in the engine may pass through a floating-point type. Percentages and rates are stored as integer basis points (`cod_fee_rate_bps`, `vat_rate_bps`), so a fee calculation is integer multiplication followed by a single defined rounding step.

Rounding is **half-up, per fee line**, exactly as approved: each fee line is rounded once, on its own, and the rounded line values are then summed into `X_e`, `X_R` and `X_r`. Totals are never rounded again, and a total is never produced by rounding an unrounded sum. `reconciliation_fee_lines` exists so this can be proved line by line for any result.

D.2 The approved formulas, unchanged

```
E  = expected collectible COD (from confirmed shipment outcome, Section C shipments)
R  = courier-reported collected COD          P_r = courier-reported payout
X_e = expected-basis deductions              X_r = deductions actually reported
X_R = deductions recalculated on the reported basis under confirmed rules

C       = R - E
delta   = X_R - X_e
V_R     = X_r - X_R
S       = P_r - (R - X_r)
V_cond  = P_r - P_cond
V_total = P_r - P_e

Identities that must hold whenever every term is present:
V_total = C - delta - V_R + S
V_cond  = S - V_R

For a cod_minus_delivery_fee base:
F_R = round_half_up( max(R - D_c, 0) x rate )
where D_c is the merchant-confirmed contractual delivery fee only.
```

Expected, courier-reported and bank-confirmed amounts are three different kinds of fact and are stored in three different places: expected values on orders, shipments and results; reported values on statement lines and results; bank-confirmed values on bank receipts and allocations. No column, view or report may substitute one for another, and no view may present a combined figure that hides which kind it is.

D.3 NULL versus verified zero

Situation in the CSV	Stored state	Stored value and effect
Fee column does not exist	`column_absent`	Value NULL for every row in the batch. The affected totals (`X_r`, and anything derived from them) are NULL, the result is `needs_review`, and a `DQ_COLUMN_ABSENT` finding is raised.
Column exists, row is blank	`blank`	Value NULL for that row only. Same downstream effect as absent, scoped to that shipment.
Fee explicitly reported as zero	`explicit_zero`	Value 0, verified. The fee counts as 0 in `X_r` and totals remain calculable.
Value invalid or unparsable	`invalid`	Value NULL, raw string preserved on the import row and the import error. Never parsed leniently into a number.
Value present and valid	`present_valid`	Value is the parsed integer poisha, greater than 0.

Three database rules protect this everywhere, not only in the parser: no money column has a DEFAULT; the state-to-value CHECK in Section C ties each state to its required nullability; and reporting queries must not use `COALESCE(amount, 0)` on any financial column - a rule enforced in code review and by the tests in Section I, because SQL cannot police its own future queries.

D.4 The contractual delivery fee: D_c, D_e and D_r stay separate

Three different delivery fees exist and are stored in three different places. `D_e`, the expected delivery fee, comes from the expected-basis fee line. `D_r`, the courier-reported delivery fee, is `statement_lines.reported_delivery_fee_poisha`. `D_c`, the contractual delivery fee, is `courier_rule_versions.contractual_delivery_fee_poisha` and is the only value that may be used as the subtractor in a `cod_minus_delivery_fee` base.

> **P0 patch rule, carried into M1 unchanged.** D_c counts as contractual only when `dc_confirmed` is explicitly true AND `dc_evidence_ref` is non-empty. A D_c value on its own, a missing flag, a false flag, or a missing evidence reference each produce `DQ_MISSING_CONTRACT`, set the affected fees and payouts (`F_e`, `F_R`, `P_e`, `P_cond`) to NULL and mark the result `needs_review`. There is no fallback to D_e or D_r, in the engine, in SQL or in a report.

Worked synthetic illustration (fictional numbers, no real courier rate): expected delivery fee D_e = 6,000 poisha, reported delivery fee D_r = 6,500 poisha, confirmed contractual fee D_c = 6,000 poisha with evidence, reported collection R = 100,000 poisha, COD fee rate 1 percent on a `cod_minus_delivery_fee` base. Then F_R = round(max(100,000 - 6,000, 0) x 1 percent) = 940 poisha. Using D_r instead would give 935 poisha and would be wrong by construction, not by arithmetic - which is why the three values may never be substituted for one another. If D_c were unconfirmed, F_R would be NULL, not 935 or 940.

D.5 Scenario variance is not a contractual finding

For a `reported_collected` COD fee base, the contractual fee is the reported-basis fee and the contractual payout comparison is against `P_cond`. `V_total` compares the reported payout with a hypothetical expected-collection scenario (`P_e`); it is information, not a contractual judgement.

Measure	What it judges	What it may affect
`C`	Collection: reported collection against expected collectible COD.	An independent collection finding. On its own it makes the overall status `open`, never `mismatch`.
`V_R`	Reported-basis fee and tax variance: what was charged against what the confirmed rules recalculate on the reported basis.	A fee/tax finding; contributes to contractual mismatch.
`S`	Whether the statement is internally consistent: does the reported payout equal reported collection minus reported deductions?	A statement-arithmetic finding; contributes to mismatch. Never forced to zero.
`V_cond`	The contractual payout comparison.	Determines contractual payout status, and therefore mismatch.
`V_total`	Scenario comparison only, for a reported_collected base.	A scenario finding, always listed, which can never on its own trigger a contractual payout mismatch.

D.6 Findings coexist; statuses are three, not one

Every discrepancy becomes its own row in `reconciliation_findings`, with its own gross amount. Findings are never netted against each other, never summed into a single loss, and never hidden because another finding exists. Derived variances (`V_cond`, `V_total`) and their underlying components (`C`, `V_R`, `S`) are displayed as an explanation chain, not added together as if they were independent losses.

Combination on one shipment	Contractual payout status	Data-quality status	Overall status
Nothing outside tolerance; all data present	matched	ok	`matched`
All findings non-zero but within tolerance	matched_with_tolerance	ok	`matched_with_tolerance` - gross amounts still shown
Under-collection only (C non-zero); V_cond = 0	matched	ok	`open` - an open collection finding, not a payout mismatch

Fee/tax variance, or S non-zero, or V_cond non-zero, outside tolerance	mismatch	ok	`mismatch`
Scenario V_total non-zero only, V_cond = 0	matched	ok	`matched`, with the scenario finding still listed (or `open` if a collection finding also exists)
Any required value NULL, missing contract, ambiguous match, or missing outcome - with or without financial findings	not_calculable or as calculated	needs_review	`needs_review` - and every financial finding that could be calculated is still stored and shown

The precedence is deterministic and narrow: `needs_review` outranks the other overall values because an incomplete picture must not be presented as settled; `mismatch` next, driven only by fee/tax, statement or contractual payout findings; `open` next, for collection-only discrepancy; then `matched_with_tolerance`; then `matched`. Precedence decides only the single overall label. It never deletes, merges or downgrades a finding, and the other two statuses are always stored alongside it.

D.7 Matching precedence

1. **Strong match:** same workspace, same courier account, and a unique normalised tracking ID match between statement line and shipment. This is the only automatic match.
2. **Duplicate or repeated tracking:** if the normalised tracking ID is not unique on either side, the line is ambiguous. It stays open for human confirmation; no automatic choice is made.
3. **Weak fallback:** blank tracking ID, an exact merchant order ID matching exactly one shipment, the same courier, and a statement date within 7 days after the shipment date where both dates exist, with no other candidate. A weak match is a proposal in `shipment_match_candidates` and takes effect only on human confirmation.
4. **Never:** no match is made on amount alone, in any code path. There is no amount value in the `match_basis` CHECK list, so the database itself rejects such a record.

Duplicate, unmatched and settlement-level lines are preserved as rows. They are never summed, merged or spread pro-rata across shipments.

D.8 Snapshots, rule versioning and evidence provenance

- **Run snapshot.** A run copies the rule definition, rule version number and content hash, the mapping snapshot, the template version, the tolerance, the engine version, and the identifiers and hashes of every input line and shipment. A result can therefore be explained years later without reference to any live, editable row.
- **Result snapshot.** Results copy E, R and P_r rather than pointing back at mutable shipment or statement values for their amounts. The foreign keys remain for navigation; the amounts on the result are the historical fact.
- **Rule versioning.** Rule versions are immutable once active. A correction creates a new version, supersedes the old one, and affects only future runs. No run is ever recalculated in place; a recalculation is a new run.
- **Evidence provenance.** Every active contractual value traces to a `courier_rule_evidence` row naming its type, reference, confirming user and time. Values with no evidence stay NULL. Public pricing pages are not evidence.
- **VAT and included tax.** An included-tax amount is a breakdown of a fee that has already been counted once; it is never subtracted again. The breakdown is calculated only when the rule states an included-tax formula (`rate_on_net` or `rate_on_gross`) and the rate. If the mode is unknown, an exact payout cannot be claimed; if the mode is known to be included but only the rate is unknown, the gross fee still counts once and only the breakdown is NULL.

D.9 Reconciliation immutability and historical referential integrity

Immutability in M1 is enforced at three levels, because the P0 prototype's application-level insert-only store is not sufficient for production. First, privileges: the runtime role has no UPDATE or DELETE grant on runs, results, findings, fee lines, statement lines, import rows or evidence. Second, RLS: no UPDATE or DELETE policy is defined for those tables, so even a granted statement would return no rows. Third, triggers or rules are not required if the first two levels are applied, and adding them is a defence-in-depth option for the implementation review, not a substitute for correct grants.

Historical integrity also constrains deletes elsewhere: foreign keys from financial history use `ON DELETE RESTRICT`. A shipment, order, rule version, template or file object that is referenced by a run cannot be deleted. Workspace closure therefore anonymises and archives rather than cascading deletes through financial tables (Section G).

D.10 Settlement and partial receipts

A settlement batch states what is expected from courier-reported figures. Bank receipts state what the merchant confirms actually arrived. Allocations connect the two, explicitly, by a human or by a deterministic rule the merchant can inspect - never by an amount match alone, and never pro-rata.

- Multiple partial receipts against one batch are normal. Each is recorded with its own amount and date.
- `bank_variance_poisha` stays NULL while any linked receipt is not explicitly marked complete, or while the batch is not fully allocated. A partial receipt produces the status `partial_receipt`, not a variance and not a discrepancy.
- Only when every linked receipt is complete and the batch is fully allocated may the variance be calculated, as allocated bank-confirmed total minus expected batch payout, and stored. Before that moment any number shown would be an invention.
- Returned shipments follow the same separation: an expected return fee, or an amount showing as owed at batch level, is not evidence that money moved. Only a receipt is evidence of settlement.

E. PostgreSQL RLS Design

Tenant isolation rests on five things working together. Any one of them on its own - including the presence of `merchant_id` - is not isolation.

1. Every tenant-owned table carries `merchant_id` and has RLS enabled and forced.
2. The application connects as a non-superuser role that does not own the tables and has no `BYPASSRLS` attribute.
3. Every transaction sets the tenant context as a transaction-local setting before it touches data.
4. Policies fail closed: with no context set, they match no rows for reads and reject writes.
5. Composite tenant foreign keys make cross-tenant relationships impossible even for a privileged writer.

> **All SQL in this section is illustrative and marked ADAPT.** Role names, schema names and grant lists must be adapted to the chosen managed PostgreSQL service, reviewed by a second engineer, and exercised by the Section I tests before any production use. Do not run it unchanged against a live database.

E.1 Roles: migration owner versus runtime

```
-- ADAPT: run as the database administrator during provisioning only.
-- The migration role owns the schema and runs Alembic.
CREATE ROLE courierledger_migrator LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE;
-- The runtime role owns nothing, cannot bypass RLS, and is the only
-- role the application is ever allowed to connect as.
CREATE ROLE courierledger_app LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE
  NOBYPASSRLS;
-- Defence in depth: assert these attributes in the Section I tests.
-- A role with BYPASSRLS, SUPERUSER, or table ownership silently
-- defeats every policy below.
```

E.2 Tenant context: transaction-local, set on every transaction

```
-- ADAPT: called by the application at the start of EVERY transaction,
-- after authentication has resolved the active workspace.
-- The third argument (true) makes the setting transaction-local:
-- it disappears at COMMIT or ROLLBACK and cannot leak to the next
-- transaction that reuses the pooled connection.
SELECT set_config('app.merchant_id', :merchant_id, true);
SELECT set_config('app.user_id',     :user_id,     true);

-- Helper used inside policies. Returns NULL when unset or malformed,
-- which makes every policy below fail closed.
CREATE OR REPLACE FUNCTION app_current_merchant_id() RETURNS uuid
LANGUAGE sql STABLE AS $$
  SELECT NULLIF(current_setting('app.merchant_id', true), '')::uuid
$$;
```

Two rules are non-negotiable. First, `SET` without `LOCAL`, and session-level `set_config(..., false)`, are forbidden in application code: with connection pooling they attach the tenant to the connection, and the next request on that connection inherits it. Second, the context is set inside the transaction that does the work, never in a separate earlier transaction on the same connection.

E.3 Enablement and policies

```
-- ADAPT: repeat for every tenant-owned table.
ALTER TABLE shipments ENABLE ROW LEVEL SECURITY;
ALTER TABLE shipments FORCE ROW LEVEL SECURITY;
-- FORCE matters: without it, the table owner is exempt from policies.

CREATE POLICY shipments_select ON shipments
  FOR SELECT TO courierledger_app
  USING (merchant_id = app_current_merchant_id());

CREATE POLICY shipments_insert ON shipments
  FOR INSERT TO courierledger_app
  WITH CHECK (merchant_id = app_current_merchant_id());

CREATE POLICY shipments_update ON shipments
  FOR UPDATE TO courierledger_app
  USING      (merchant_id = app_current_merchant_id())
  WITH CHECK (merchant_id = app_current_merchant_id());
-- The WITH CHECK on UPDATE prevents moving a row to another tenant.

CREATE POLICY shipments_delete ON shipments
  FOR DELETE TO courierledger_app
  USING (merchant_id = app_current_merchant_id());
```

For append-only tables (runs, results, findings, fee lines, statement lines, import rows, import errors, evidence, audit logs) there is deliberately **no UPDATE policy and no DELETE policy**, and no corresponding grant. The absence is the control: an UPDATE from the runtime role affects zero rows and a DELETE is denied, which the tests assert explicitly. For `merchants` itself, the SELECT policy is membership-based - a user may see a workspace only where an active membership row links them - and workspace creation runs through a controlled function rather than a general INSERT policy. For `users` and `couriers`, which are not merchant-scoped, access is controlled by grants and application authorisation, and that difference is stated rather than disguised as an RLS policy.

E.4 Grants: least privilege per table class

```
-- ADAPT: generate per table from the classification in Section C.
-- Mutable tenant tables:
GRANT SELECT, INSERT, UPDATE, DELETE ON shipments
  TO courierledger_app;
-- Append-only financial history:
GRANT SELECT, INSERT ON reconciliation_runs,
  reconciliation_results, reconciliation_fee_lines,
  reconciliation_findings, statement_lines
  TO courierledger_app;
-- Reference data:
GRANT SELECT ON couriers TO courierledger_app;
-- No grant of any kind on any table to PUBLIC.
-- No TRUNCATE, no REFERENCES, no TRIGGER for the runtime role.
```

The single permitted exception is the run status transition (`running` to `completed` or `failed`, plus `completed_at`). Implement it either as a column-level UPDATE grant on those two columns with an UPDATE policy restricted to them in effect, or as a `SECURITY DEFINER` function owned by the migrator that validates the run's merchant against the caller's context. The function route is recommended: it keeps the table grant clean and gives one auditable place for the transition. Either way, no other column of a run may ever be updated.

E.5 Missing context, pooling and bypass prevention

Situation	Required behaviour
Tenant context not set	`app_current_merchant_id()` returns NULL. SELECT returns zero rows, INSERT fails its WITH CHECK, UPDATE and DELETE affect zero rows. The application treats zero rows on an expected read as an error and logs it; it never retries without context or falls back to an unscoped query.

Context set to a malformed value	The cast in the helper fails the transaction. That is intentional: a malformed context is a programming error and must be loud, not a silent empty result. The application validates the UUID before setting it.
Pooled connection reused by another tenant	Safe only because the setting is transaction-local. Additionally, the pool must be configured in transaction mode if an external pooler is used (for example PgBouncer transaction pooling), and the application must never rely on session state, temporary tables or prepared-statement side effects carrying tenant information.
Background or scheduled work	Jobs have no ambient tenant. Each job item sets the context for its own merchant inside its own transaction. A job that processes several merchants uses one transaction per merchant, never one transaction with a changing context and cached rows.
Administrative and support access	No standing bypass role for support. Break-glass access, if ever approved, is a separate audited procedure outside the application - it is not a policy in this design, not a flag column, and not a second application role.
Migrations	Alembic connects as the migrator, which owns the tables. Because FORCE is enabled, the owner is still subject to policies for row operations; migrations that must touch rows across tenants do so in DDL-focused migrations or with an explicitly reviewed, temporary policy change recorded in the migration itself. Data migrations must name the tenant they affect.

A further guard: application code must never build a query against a tenant table without going through the tenant-aware repository layer in Section H, and the test suite includes a check that every tenant-owned table has RLS enabled, forced, and at least a SELECT and INSERT policy - so a future migration that adds a table and forgets its policies fails the build instead of shipping an open table.

F. Indexing and Performance

The indexing rule for M1 is simple: index the tenant-first query patterns the product actually runs, and nothing speculative. Every tenant query begins with `merchant_id`, so every useful index begins with it too - which also makes each index serve the RLS policy predicate.

F.1 Query patterns and their indexes

Query pattern	Index
Results for one run, paged	`reconciliation_results (merchant_id, run_id, id)`
Review queue: open, mismatch and needs_review results	Partial: `reconciliation_results (merchant_id, overall_status, created_at) WHERE overall_status IN ('open','mismatch','needs_review')`
Findings for a result; findings by code for a merchant	`reconciliation_findings (merchant_id, result_id)`; `(merchant_id, finding_code, created_at)`
Shipment lookup by tracking ID during import	`shipments (merchant_id, tracking_id_normalised)` - non-unique on purpose
Statement lines needing matching	Partial: `statement_lines (merchant_id, import_batch_id) WHERE match_status IN ('unmatched','ambiguous')`
Import rows for a batch, in file order	Unique `import_rows (merchant_id, import_batch_id, row_number)` as the constraint; it doubles as the access index
One active COD-bearing shipment per order	Partial unique `shipments (merchant_id, order_id) WHERE is_cod_bearing AND NOT is_superseded`
Only one confirmed match per statement line	Partial unique `shipment_match_candidates (statement_line_id) WHERE decision = 'confirmed'`
Duplicate file detection	Unique `file_objects (merchant_id, sha256)`
Determinism and duplicate-run detection	Unique `reconciliation_runs (merchant_id, run_hash)`
Settlement work list	`settlement_batches (merchant_id, allocation_status, period_start)`; `bank_receipts (merchant_id, settlement_batch_id)`; `settlement_allocations (merchant_id, bank_receipt_id)`
Audit review	`audit_logs (merchant_id, created_at)`; `(merchant_id, action, created_at)`

F.2 What is deliberately not indexed or built

- No full-text index, no trigram index and no JSONB GIN index in M1. `raw_values` and snapshot columns are read by key, not searched. If support later needs raw-row search, that is a measured addition, not a guess now.

- No partitioning in M1. `import_rows` and `audit_logs` are the first candidates if row counts ever justify it, and both are already keyed by merchant and time in a way that would allow it later.
- No materialised totals. Batch and dashboard totals are computed from fee lines and results, or maintained in the same transaction that writes allocations. A cached total that can drift from its lines is a financial risk, not a performance feature.
- No index on amount columns. Nothing legitimate searches by amount, and an amount index would invite the amount-based matching this design forbids.

F.3 Scaling notes

SME statement volumes are modest: the design target is correctness under concurrent imports by one merchant and steady use by many small merchants, not bulk throughput. The realistic growth pressures are `import_rows` volume, audit volume and the number of tenants sharing one database. The responses, in order, are: retention and purging of raw file bytes (Section G), archival of audit logs, and only then partitioning or a per-tenant schema discussion - which would be a new architecture decision, not an optimisation. Any query that cannot use a tenant-first index should be treated as a bug.

G. Data Lifecycle and Retention

> **No retention period in this section is a legal claim.** Every duration below is a proposed operational default marked **OPEN - POLICY AND LEGAL REVIEW REQUIRED**. Bangladeshi legal, tax and accounting advice must be taken before any period is adopted, and the adopted periods must be recorded as a policy decision separate from this specification.

Data	Proposed lifecycle
Uploaded CSV files (object storage)	Retained while the import batch and its runs are retained, so a result can always be traced to its source file. Byte-level purge after workspace closure or after the adopted retention period, leaving `file_objects` metadata and hash in place so historical references still resolve. **OPEN - POLICY AND LEGAL REVIEW REQUIRED**
Raw import rows	Retained with their batch. They are the evidence for every normalised value and must not be deleted while the related runs exist. Any future archival moves them whole, with their batch, never selectively by row.
Failed imports	Kept, with their errors, for the same period as successful imports. A failed import is diagnostic evidence; deleting failures would hide systematic template or statement problems. Batches that were uploaded and abandoned before mapping may be purged earlier under a housekeeping rule. **OPEN**
Reconciliation runs, results, findings, fee lines and snapshots	Never deleted in normal operation. Corrections create new runs. This history is the product's core evidence and its retention is a policy decision, not a storage optimisation. **OPEN - POLICY AND LEGAL REVIEW REQUIRED**
Audit logs	Append-only, retained at least as long as the financial history they describe. Archival to cold storage is acceptable; selective deletion is not. **OPEN - POLICY AND LEGAL REVIEW REQUIRED**
User sessions	Expired and revoked sessions purged on a short housekeeping cycle (proposed 30 days after expiry). **OPEN**
Idempotency keys	Purged after their expiry window (OD-13). They are operational records, not financial history.
User deletion	Deleting a user removes their sign-in credentials and personal profile fields, and revokes their sessions and memberships. Their identity remains as an anonymised actor reference in audit logs and as `created_by` / `confirmed_by` attributions on financial records, because removing those would break provenance. The exact anonymisation form is a policy decision. **OPEN - POLICY AND LEGAL REVIEW REQUIRED**
Merchant workspace deletion	A two-stage process: closure (status `closing`, access revoked, exports offered to the owner) followed by erasure after the adopted retention and any legal hold. Erasure deletes personal data and stored file bytes, and anonymises merchant-identifying fields. Whether financial records themselves are then deleted or retained in anonymised form is the single most consequential open decision in this section and must not be decided by an engineer alone. **OPEN - POLICY AND LEGAL REVIEW REQUIRED**
Synthetic and demo data	Demo data is created only through the `source = 'demo_synthetic'` path, in clearly labelled demo workspaces, and is purged with the workspace. Synthetic data must never be mixed into a real merchant's workspace.

H. SQLAlchemy 2.x and Alembic Design

This section describes organisation and patterns only. It is not an application implementation.

H.1 Model organisation

- One models module per domain module - identity, courier configuration, orders, import, reconciliation, settlement, audit - mirroring the modular monolith. A shared base module holds the declarative base, the tenant mixin and the timestamp mixin.
- The tenant mixin contributes `merchant_id`, the composite `UniqueConstraint(merchant_id, id)` and the standard tenant indexes, so a model cannot quietly omit them.
- Money columns are declared as `BigInteger` with the `_poisha` suffix enforced by a naming convention and a lint check over the metadata. Rate columns are `Integer` basis points. No model may declare a float type for a financial field.
- Enumerations are stored as TEXT with database CHECK constraints, not as PostgreSQL ENUM types: CHECK lists can be extended in an ordinary migration, while altering an ENUM type is more restrictive and harder to roll back. The Python side mirrors the same closed lists.
- JSONB is used only where the content is genuinely a snapshot or a raw record: `raw_values`, `mapping_snapshot`, `rule_snapshot`, `input_snapshot`, `rule_definition` and audit `detail`. Anything queried, joined, constrained or summed gets a real column.

H.2 Relationship strategy

- Relationships are defined for navigation within a module. Cross-module access goes through the owning module's repository or service, not through a lazy relationship that can wander across the whole graph.
- No lazy loading in request paths. Queries state their joins and eager-loading explicitly, so the SQL a reviewer sees is the SQL that runs - which matters when the reviewer is checking that a query is tenant-scoped.
- Cascades are set to `RESTRICT` semantics at the database level for anything referenced by financial history. ORM-level cascade deletes are disabled for those models; deletion is a workflow, not a side effect of removing an object.
- Composite foreign keys are declared explicitly in the models so that the ORM and Alembic generate the same constraints the ERD shows. A model whose foreign key is single-column where Section C requires a composite is a review failure.

H.3 Tenant-aware repository pattern

- There is no repository method that runs without a tenant. Every repository is constructed with a session that already carries a resolved tenant context object, and its methods take no optional merchant parameter - an optional parameter is exactly how an unscoped query gets written by accident.
- One unit-of-work boundary per request: open the transaction, set the tenant context with `set_config(..., true)`, do the work, commit. The context is never set outside a transaction and never reused across transactions.
- The ORM session must not use a connection that another request's transaction has touched without the context being reset by the transaction boundary itself. Session-level state, including any cached identity map, is per-request.
- Queries still filter by `merchant_id` explicitly even though RLS would filter anyway. Defence in depth is deliberate: the explicit predicate makes intent visible in review and keeps query plans on the tenant-first indexes, while RLS remains the guarantee if a predicate is ever missed.
- Admin or cross-tenant operations do not exist in the repository layer. If a genuine operational need appears, it is a separate reviewed tool with its own role and audit trail, not a boolean flag on a repository.

H.4 Alembic migration structure

- Migrations connect as the migrator role, never as the runtime role. The runtime role is granted privileges by a dedicated, reviewed migration (or provisioning step), not by application startup code.
- One migration concern per revision: schema, then constraints, then indexes (built concurrently where the table may already hold data), then RLS enablement and policies, then grants. Separating RLS into its own revision makes the most safety-critical change reviewable on its own.
- Every migration that creates a tenant-owned table must, in the same revision, add its `merchant_id`, its composite unique constraint, its indexes and its RLS policies. A table that exists for one revision without policies is an open table.
- Autogenerate is a drafting aid only. Every autogenerated revision is reviewed line by line for CHECK constraints, partial indexes, composite keys and policy statements, none of which autogenerate reliably produces.
- Downgrades are written for structural migrations. Data and evidence migrations state plainly when they are irreversible rather than pretending a downgrade exists.
- A CI check inspects the final metadata and the live test database: every tenant table has RLS enabled and forced, the runtime role is neither owner nor superuser and has no BYPASSRLS, and no money column has a floating type or a zero default. That

check is part of the Section I suite.

H.5 Database constraints complement application validation

The division of labour is deliberate. The database guarantees the invariants that must hold no matter who writes the row: types, nullability, state-to-value consistency, tenant consistency of relationships, uniqueness, one active COD-bearing shipment per order, one confirmed match per line, and the identities between a result's stored variances when all terms are present. The application handles everything that needs context the database does not have: whether evidence is genuinely merchant confirmation, whether a statement template matches a file, how findings are explained in English and Bangla, and how a user is guided to fix a problem. Where a rule can be expressed in both places, it is expressed in both, and a disagreement between them is a bug to fix, not a choice to make at runtime.

I. Database Testing Strategy

All scenarios run against a real PostgreSQL instance with the real migrations applied and with connections made as the runtime role, unless a scenario says otherwise. Tests that run as the migration owner prove nothing about RLS. Test data is synthetic only. Each scenario names its expected result; a scenario that cannot state its expected result is not yet a test.

ID	Scenario	Setup and action, then expected result
T-01	Cross-tenant read	Create merchants A and B with shipments in each. As the runtime role with context A, select all shipments. Expected: only A's rows. Repeat with an explicit `WHERE merchant_id = B` predicate. Expected: zero rows, not B's rows.
T-02	Cross-tenant write and tenant move	With context A, attempt to insert a shipment with merchant B, and to update one of A's shipments to merchant B. Expected: insert rejected by WITH CHECK; update affects zero rows or is rejected. A's row still belongs to A.
T-03	RLS enforcement on every tenant table	Enumerate tenant-owned tables from the catalog. For each, assert RLS is enabled and forced, and that SELECT and INSERT policies exist for the runtime role. Expected: no table missing. This test is the guard against a future migration shipping an open table.
T-04	Missing tenant context	Open a transaction as the runtime role, set no context, and attempt select, insert, update and delete on a tenant table. Expected: select returns zero rows, insert fails, update and delete affect zero rows. The application-level test asserts this is surfaced as an error, not as an empty list presented to a user.
T-05	Cross-tenant foreign key	With context A, insert a shipment for an order that belongs to B, and a statement line pointing at B's shipment. Expected: both rejected by the composite foreign key, independent of RLS.
T-06	Pool tenant leakage	Use a pool of size 1. Run a transaction for A that sets context and commits; then a transaction for B on the same connection; then a transaction with no context. Expected: each sees only its own tenant, and the context-free transaction sees nothing. Repeat with rollback instead of commit, and with an exception mid-transaction.
T-07	Runtime role cannot bypass	Assert the runtime role's attributes: not superuser, no BYPASSRLS, not owner of any tenant table, no membership in the migrator role. Attempt DDL as the runtime role. Expected: attribute assertions pass and DDL is denied.
T-08	Duplicate tracking IDs	Import a synthetic statement where one tracking ID appears twice, and where two shipments share a tracking ID. Expected: all rows preserved, no automatic match, both lines flagged ambiguous with a duplicate-tracking finding, and no amounts summed.
T-09	Concurrent imports	Run two imports of different files for the same merchant concurrently, and the same file twice concurrently. Expected: different files both complete with correct row counts; the identical file is detected by hash or idempotency key and does not create a second set of statement lines.
T-10	Idempotency	Replay an import-create and a run-create request with the same key and the same request hash. Expected: the original object is returned, no duplicate exists. Replay with the same key and a different hash. Expected: an explicit error.
T-11	Immutable reconciliation runs	Create a run, record a hash of its run, result, finding and fee-line rows. Attempt UPDATE and DELETE on each as the runtime role; then create a second run over corrected data. Expected: update and delete are denied or affect zero rows; the first run's rows are byte-identical afterwards; the second run is a separate row.
T-12	Deterministic output	Run the same batch twice with the same rule version, tolerance and engine version. Expected: identical run_hash handling (either deduplicated by the unique constraint or two runs with identical computed values), and identical results, findings and fee lines in every amount and status.
T-13	Rule version immutability	Activate a rule version and reconcile with it. Attempt to update its rate and its D_c in place; then create a corrected version. Expected: in-place update of an active version is blocked by the write path and the version is superseded instead; the earlier run still shows the original rule snapshot and hash.
T-14	D_c confirmation	Four cases for a cod_minus_delivery_fee base: no confirmation flag; flag false; flag true with empty evidence; flag true with evidence. Expected: the first three produce DQ_MISSING_CONTRACT, NULL fees and payouts and needs_review, with no substitution of D_e or D_r; the fourth calculates normally.

T-15	NULL versus zero, all five states	Import synthetic rows covering column absent, blank, invalid, explicit zero and valid value for one fee. Expected: states and values exactly as Section D specifies; X_r is NULL for the first three, includes 0 for explicit zero, and no row's missing value appears anywhere as 0.
T-16	Classification precedence	Construct results with: collection-only variance; scenario-only V_total variance; fee variance plus under-collection; and a financial variance plus missing data. Expected overall statuses open, matched (with scenario finding listed), mismatch, and needs_review respectively, with every finding present in each case and contractual payout status decided by V_cond alone.
T-17	Partial settlement allocations	Create a batch, add one partial receipt, allocate it in part, then add a second receipt and complete allocation, marking receipts complete only at the end. Expected: bank variance is NULL at every intermediate step, allocation status moves through partially_allocated to fully_allocated, over-allocation beyond a receipt's amount is rejected, and a variance appears only after completion plus full allocation.
T-18	One active COD-bearing shipment	Attempt to create a second active COD-bearing shipment for one order, and a replacement after superseding the first. Expected: the second active shipment is rejected by the partial unique index and flagged for review; the superseding flow succeeds and the superseded shipment remains queryable.
T-19	Matching never uses amount	Provide a statement line whose amount matches a shipment exactly but whose tracking ID and order reference match nothing. Expected: no candidate is created, the line stays unmatched, and inserting a candidate with an amount basis is rejected by the CHECK constraint.
T-20	Financial identities	For every result in a synthetic suite covering delivered, partial, returned and cancelled shipments, assert V_total = C - delta - V_R + S and V_cond = S - V_R whenever all terms are present, and assert that a NULL term forces its dependents to NULL.

Suite placement: T-01 to T-07 and T-11 are release blockers for any change touching tenancy, roles or migrations. The financial scenarios (T-14 to T-20) are release blockers for any change touching the engine, import parsing or rule handling. None of these tests is optional, and none may be run against a database where the test role owns the tables.

J. Open Decisions

Each decision states why it matters, the realistic options, a recommended default, and whether it blocks implementation. "Blocks" means database implementation should not start on the affected tables until the decision is recorded; it does not mean the whole design is blocked.

ID	Decision	Why it matters / options	Recommended default	Blocks?
OD-01	Gate: market validation and production-readiness approval	This design exists so M1 can be reviewed, but building it before the Day 7 and Day 14 validation gates would repeat the drift the planning phase was designed to prevent.	No implementation until both gates and explicit approval are recorded.	Yes - everything
OD-02	Managed PostgreSQL provider and version	Affects available extensions, pooling behaviour, backup model and whether FORCE RLS and exclusion constraints behave as assumed. Options: a major managed provider, a Bangladesh-accessible regional provider, or self-managed in Docker for early validation only.	Managed PostgreSQL, current stable major version, region chosen for latency from Bangladesh, transaction-mode pooling verified by T-06.	Yes - provisioning
OD-03	Identity: built-in credentials or external provider	Decides whether user_sessions holds refresh hashes or only a revocation list, and whether password_hash is ever populated.	External provider if it is already available at no extra cost; otherwise built-in with hashed credentials and the session table as specified. Never store raw tokens either way.	Partial - identity tables
OD-04	Run status transition mechanism	Column-level grant versus a SECURITY DEFINER function. Affects how strictly append-only the run table can be.	SECURITY DEFINER function, owned by the migrator, validating tenant context.	No
OD-05	Object storage for uploaded files and evidence	file_objects assumes bytes live outside PostgreSQL. Provider choice affects storage_key format, access control and purge mechanics.	Private bucket, opaque keys, signed access only through the application. Bytes never in the database.	Partial - import

OD-06	Retention and erasure policy	Section G proposes lifecycles but no lawful period. Needs Bangladeshi legal and tax advice, especially for financial history versus workspace erasure.	Retain financial history, anonymise personal data on erasure, purge file bytes - subject to legal confirmation before launch.	Yes - launch, not build
OD-07	Are statement templates global or merchant-owned?	Global templates are simpler and consistent; merchant-owned templates allow for per-seller statement variants but need merchant_id, RLS and a sharing model on two more tables.	Global per courier, versioned, with per-import mapping snapshots. Add merchant overrides only if validation shows sellers receive materially different layouts.	No
OD-08	Are merchant order references unique per merchant?	Decides whether (merchant_id, merchant_order_ref) is a unique constraint or only an index. A wrong uniqueness claim would reject real duplicate references.	Index only, not unique, until seller evidence shows references are reliably unique.	No
OD-09	What receipt reference may be stored?	Traceability versus the absolute ban on storing bank account numbers. Even partial digits are a policy choice, not an engineering convenience.	Merchant-supplied label only, no digits of any account number, unless a written policy decision permits last-four storage.	Partial - settlement
OD-10	Rule activation and overlap model	Whether one active version per account is enforced by a partial unique index, or overlapping effective windows are allowed with an exclusion constraint.	One active version per account, activated in a single transaction that supersedes the predecessor. Effective windows recorded for history.	No
OD-11	Guaranteeing at least one active owner per workspace	Not expressible as a plain constraint. Options: application rule with a locking transaction, or a deferred constraint trigger.	Application rule inside a transaction that locks the membership rows for the workspace, plus an audit entry.	No
OD-12	Can a statement line legitimately report a negative net payout?	Return-heavy statements might. Clamping to zero would fabricate a value; allowing negatives affects CHECK constraints on reported payout and on result amounts.	Allow negative reported payout where the template and merchant evidence support it, flagged for review; keep all other reported amounts non-negative. Confirm with real redacted statement examples during validation.	Partial - import checks
OD-13	Idempotency key retention window	Too short invites duplicate runs on slow retries; too long grows an operational table.	7 days for import and run creation, then purge.	No
OD-14	Allocation sum enforcement mechanism	Cross-row sums cannot be a plain CHECK. Options: transactional enforcement with row locks, plus a deferred constraint trigger as backstop, or transactional enforcement alone.	Transactional enforcement with row locks, plus the deferred trigger if the implementation review accepts its complexity.	No
OD-15	Additional fee types beyond the closed fee_code list	Real statements may carry fee types not yet seen. Adding a code is a specification change because it affects X_e, X_R and X_r composition.	Keep the list closed in M1. Unrecognised fee columns are preserved in raw rows, reported as a data-quality warning, and never folded into a total.	No

Appendix 1 - Financial invariant traceability

Invariant from the approved specification	Where it is enforced	Test
Money is integer poisha; no floating-point money	BIGINT columns throughout Section C; basis-point rates; CI metadata check	T-20, CI check in H.4
Missing is NULL, never zero; explicit zero is distinguishable	Per-field state columns and state-to-value CHECKs on statement_lines and fee lines	T-15
Raw and normalised values preserved separately	import_rows.raw_values; raw and normalised tracking and fee fields on shipments and statement_lines	T-15, T-08
Runs preserve immutable input and rule snapshots	reconciliation_runs snapshot columns, run_hash, SELECT/INSERT-only grants, no UPDATE/DELETE policy	T-11, T-12
Rule versions historically traceable and immutable	courier_rule_versions versioning plus courier_rule_evidence provenance	T-13
Corrections create new records or runs	Append-only classes in Section C; superseded shipments and reversed allocations	T-11, T-18
Findings independent, never netted, never labelled confirmed losses	reconciliation_findings as rows with gross amounts; no loss column or view	T-16
Partial bank receipts supported	bank_receipts (many per batch) plus settlement_allocations	T-17
Bank variance NULL until receipt complete and batch fully allocated	settlement_batches.bank_variance_poisha write rule in D.10	T-17
No bank account numbers stored	No such column exists; audit detail rule; OD-09	Schema review, T-03 style catalog check
One active COD-bearing shipment per order	Partial unique index on shipments	T-18
Superseded shipments retained but excluded from active reconciliation	is_superseded flags, superseded_by link, partial indexes	T-18
Weak matching never on amount alone	match_basis CHECK list has no amount value; candidate workflow in D.7	T-19
Unknown contractual rules never inferred	NULL rule fields, DQ_MISSING_CONTRACT, evidence requirement for D_c	T-14
D_c requires explicit confirmation and evidence	dc_confirmed plus dc_evidence_ref CHECKs on courier_rule_versions	T-14
Scenario V_total cannot trigger a contractual mismatch	Separate status columns and derivation rules in D.5 and D.6	T-16
Per-line half-up rounding	reconciliation_fee_lines per component, totals as sums of rounded lines	T-20

Appendix 2 - Glossary

Term	Meaning in this document
Poisha	The smallest unit used for money here: 100 poisha = 1 taka. All amounts are whole poisha integers.
E, R, P_r	Expected collectible COD, courier-reported collected COD, and courier-reported payout for one shipment.
X_e, X_r, X_R	Total deductions on the expected basis, as actually reported, and recalculated on the reported basis under confirmed rules.
D_e, D_r, D_c	Expected, courier-reported and merchant-confirmed contractual delivery fees. Three separate values, never substituted.
C, V_R, S, V_cond, V_total	Collection variance, reported-basis fee/tax variance, statement inconsistency residual, conditional (contractual) payout variance, and scenario payout variance.
Finding	One independent, separately stored discrepancy or data-quality problem, with its own gross amount.

RLS / FORCE	Row-Level Security: per-row access policies enforced by PostgreSQL. FORCE makes the policies apply to the table owner as well.
Tenant	One merchant workspace. Identified everywhere by merchant_id.

Appendix 3 - Review checklist

- Every table in the ERD appears in Section C with the same name, keys and relationships, and every Section C table appears in the ERD.
- No money column uses a floating type, has a zero default, or lacks the _poisha suffix.
- Every tenant-owned table has merchant_id, a composite unique (merchant_id, id), tenant-first indexes, RLS enabled and forced, and policies for the operations its class permits.
- Every append-only table has no UPDATE or DELETE grant and no UPDATE or DELETE policy for the runtime role, except the single documented run-status transition.
- D_c, dc_confirmed and dc_evidence_ref behave exactly as the P0 patch requires, and no code path falls back to D_e or D_r.
- Bank variance cannot be written before receipt completion and full allocation.
- No column, in any table, stores a bank account number, a raw credential or a raw session token.
- All Section J decisions marked as blocking are recorded before the affected implementation begins.

> **End of specification.** This document is a design blueprint for review. It authorises no implementation, migration, deployment or account change. M1 development remains gated behind market validation and explicit production-readiness approval.